PARENT JSON


```
{
  "spec_version": "1.0",
  "spec_name": "MIMIR_LinkedIn_Post_Generator",
  "description": "LLM spec for generating LinkedIn posts using MIMIR's doctrine. Input aware, agnostic by default, with post types, skeletons, templates, hooks, and execution logic.",
  "inputs_schema": {
    "description": "Optional inputs. Use what is present. Stay agnostic for missing fields.",
    "fields": {
      "topic": {
        "type": "string",
        "required": true,
        "description": "Short phrase or sentence about what the post should be about."
      },
      "core_insight": {
        "type": "string",
        "required": false,
        "description": "The main idea, truth, or lesson the user wants to convey. If empty, infer from topic."
      },
      "persona": {
        "type": "string",
        "required": false,
        "description": "Target reader role. Examples: SDR, AE, CRO, Founder, Product Manager, CMO, Leader. If missing, treat as general business professional."
      },
      "industry": {
```

"type": "string",

"required": false,

"description": "Industry of the reader or use case. Examples: SaaS, Travel, Retail, Fintech, Manufacturing. If missing, avoid industry specific references."

},

"geography": {

"type": "string",

"required": false,

"description": "Region shorthand. Examples: US, EU, India, APAC, LatAm. If missing, keep tone globally neutral."

},

"tone": {

"type": "string",

"required": false,

"description": "Desired tone. Examples: sharp, reflective, teaching, contrarian, neutral. If missing, default to simple, direct, reflective."

},

"language": {

"type": "string",

"required": false,

"description": "Output language. Allowed: en, en_hinglish, es. Default is en (simple global English)."

},

"challenge": {

"type": "string",

"required": false,

```
      "description": "Specific problem or challenge. Examples: demo nerves, lost deals, leadership clarity, low pipeline. If missing, keep challenge framing general."
    }
   },
   "agnostic_rules": {
    "description": "How to behave when some inputs are missing.",
    "if_missing_persona": "Write for a general business professional. No role specific jargon.",
    "if_missing_industry": "Use universal examples such as clarity, friction, decisions, communication, ownership.",
    "if_missing_geography": "Use globally neutral English and avoid regional references.",
    "if_missing_tone": "Use simple, direct, reflective tone.",
    "if_missing_language": "Assume en and use plain global English.",
    "if_missing_core_insight": "Infer a simple insight from the topic but keep it broad and easy to agree with.",
    "asking_policy": "Do not ask the user follow up questions for missing inputs unless the user explicitly requests input collection (for example: 'ask me' or 'collect inputs')."
   }
  },
  "post_types": {
   "description": "Main post types with when to use and section framing.",
   "narrative": {
    "id": "PT_NARRATIVE",
    "description": "Story driven, lesson from a moment.",
    "use_when": [
     "Topic is about a lesson learned.",
     "User mentions story, experience, or specific incident."
    ],
```

```
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
"jolt": {
  "id": "PT_JOLT",
  "description": "Shock or wake up style post that hits directly.",
  "use_when": [
    "Tone is sharp.",
    "User wants to wake people up or call out a painful mistake."
  ],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
"insight": {
  "id": "PT_INSIGHT",
  "description": "Deep explanation of a pattern or truth.",
  "use_when": [
    "User wants to explain something in depth.",
    "Topic is about frameworks, patterns, or principles.",
    "If unclear, this is the safest default."
  ],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
"contrarian": {
  "id": "PT_CONTRARIAN",
  "description": "Flips a common belief or crowded opinion.",
  "use_when": [
    "User clearly wants to challenge a popular belief.",
```

```
      "Topic starts with why everyone is wrong, or flips a norm."
    ],
    "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
   },
   "teaching": {
    "id": "PT_TEACHING",
    "description": "Shows how to do one thing better.",
    "use_when": [
     "Topic is about improving a skill.",
     "User mentions steps, practice, or coaching."
    ],
    "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
   }
  },
  "templates": {
   "description": "Ready made section prompts for each post type.",
   "items": [
    {
     "id": "T_NARRATIVE",
     "post_type": "narrative",
     "description": "Story first, then lesson.",
     "section_prompts": {
      "hook": "One short line that exposes tension or confusion from the moment.",
      "context": "Describe where you were, who was involved, or what was happening in 2 to 4 short lines.",
      "insight": "Write the lesson learned in a clear way. Show what changed in your thinking.",
```

"story": "Replay the key moment or small dialogue that brought the insight alive.",

"consequence": "Explain what goes wrong when people ignore this lesson. Show personal and business effect if applicable.",

"shift": "Describe one simple change in thinking or behavior that avoids the mistake.",

"close": "End with a quiet question or reflection that lets the reader apply this to their own life."

}

},

{

"id": "T_JOLT",

"post_type": "jolt",

"description": "Short, sharp, consequence heavy.",

"section_prompts": {

"hook": "Write a punchy truth that directly names the painful problem.",

"context": "Describe a very common scene where this problem appears.",

"insight": "Explain the hidden cause behind the problem in simple language.",

"story": "Give one quick example that readers can picture in their mind.",

"consequence": "State the real cost of ignoring this. Show emotional and practical damage.",

"shift": "Offer one firm corrective move the reader can start immediately.",

"close": "End with a short line or question that lets the reader sit with the discomfort."

}

},

{

"id": "T_INSIGHT",

"post_type": "insight",

"description": "Explains a pattern with clarity.",

```
  "section_prompts": {
    "hook": "State a clear pattern or problem that many people face.",
    "context": "Describe when and where this pattern usually shows up.",
    "insight": "Break down the real reason this pattern exists. Use simple cause and effect.",
    "story": "Share a small example or real case that makes the insight feel real.",
    "consequence": "Explain what happens over time if this pattern stays unchanged.",
    "shift": "Offer one fresh way to look at the pattern that opens a better path.",
    "close": "Ask the reader to think of one place in their life where this shows up."
  }
},
{
  "id": "T_CONTRARIAN",
  "post_type": "contrarian",
  "description": "Challenges standard advice.",
  "section_prompts": {
    "hook": "Write one line that clearly contradicts a common belief.",
    "context": "Explain why most people believe the opposite.",
    "insight": "Show the flaw in that common belief and explain your view.",
    "story": "Use a short story or example where the common belief failed.",
    "consequence": "Describe the damage that belief causes when followed blindly.",
    "shift": "Present your alternative belief or rule as a simpler, stronger option.",
    "close": "Invite the reader to test your view in one real situation."
  }
},
{
  "id": "T_TEACHING",
```

```
    "post_type": "teaching",
    "description": "Teaches a specific skill or move.",
    "section_prompts": {
      "hook": "State the problem the reader is facing in one sharp line.",
      "context": "Describe where and how this problem appears in their day.",
      "insight": "Explain the real root cause of the problem in clear terms.",
      "story": "Give a quick micro example that shows the problem in action.",
      "consequence": "Spell out what continues to go wrong if this skill does not improve.",
      "shift": "Give one clear, small, practical move the reader can try today.",
      "close": "End with an invitation to test this move once and notice the difference."
    }
   }
  ]
 },
 "skeletons": {
  "description": "50 high level skeletons. Each has a name, simple use rules, and section order. All use the common layout: Hook, Context, Insight, Story, Consequence, Shift, Close.",
  "items": [
   {
    "id": "S01",
    "name": "Hidden Cost",
    "use_when": ["Topic about unseen losses or slow damage."],
    "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
   },
   {
    "id": "S02",
```

```
  "name": "Everyone Gets This Wrong",
  "use_when": ["Topic about common mistake or myth."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S03",
  "name": "Bought Lesson",
  "use_when": ["Topic about an expensive mistake that taught something important."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S04",
  "name": "Daily Blind Spot",
  "use_when": ["Topic about something people overlook in daily work or life."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S05",
  "name": "One Moment That Changes Everything",
  "use_when": ["Topic about a turning point, decision, or key moment."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S06",
  "name": "Why You Feel Stuck",
  "use_when": ["Topic about feeling stuck, blocked, or stalled."],
```

```
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S07",
  "name": "What You Think vs What Actually Happens",
  "use_when": ["Topic about gap between belief and reality."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S08",
  "name": "Quiet Skill That Changes Outcomes",
  "use_when": ["Topic about underrated skill or habit."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S09",
  "name": "Nobody Tells You This Early",
  "use_when": ["Topic about truths people learn late in their career or life."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S10",
  "name": "Stop Doing X If You Want Y",
  "use_when": ["Topic about tradeoff or wrong path people keep choosing."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
```

```
{
  "id": "S11",
  "name": "Silence Speaks Louder",
  "use_when": ["Topic about non verbal behavior or unspoken signals."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S12",
  "name": "Small Action Big Signal",
  "use_when": ["Topic about small acts that reveal big truths."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S13",
  "name": "Pain You Should Not Ignore",
  "use_when": ["Topic about warning signs or early pain points."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S14",
  "name": "One Line That Changed My Thinking",
  "use_when": ["Topic about a single sentence or advice that shifted perspective."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
```

```
  "id": "S15",
  "name": "Skill vs Nerves",
  "use_when": ["Topic about performance under pressure and nerves vs skill."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
 },
 {
  "id": "S16",
  "name": "Reps Ask The Wrong Question",
  "use_when": ["Topic about misdirected questions or focus in sales or work."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
 },
 {
  "id": "S17",
  "name": "Buyer Already Decided Here",
  "use_when": ["Topic about early decision moments in buyer journey or stakeholder alignment."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
 },
 {
  "id": "S18",
  "name": "Your Tone Reveals Your Intent",
  "use_when": ["Topic about tone, energy, or delivery revealing true intent."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
 },
 {
  "id": "S19",
```

```
  "name": "You Lost Before The Demo",
  "use_when": ["Topic about upstream mistakes before key events like demos or meetings."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S20",
  "name": "Lesson Behind The Conflict",
  "use_when": ["Topic about conflict moments teaching important lessons."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},

{
  "id": "S21",
  "name": "Hard Truth About Leadership",
  "use_when": ["Topic about leaders, teams, culture, or ownership at the top level."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S22",
  "name": "Metrics Everyone Trusts But Should Not",
  "use_when": ["Topic about misleading metrics or vanity numbers."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S23",
  "name": "Stop Blaming The Tool",
```

```
  "use_when": ["Topic about skill vs tools, and misdirected blame."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S24",
  "name": "Skill You Need But Never Practice",
  "use_when": ["Topic about important but neglected skills."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S25",
  "name": "Gap Between Thinking And Doing",
  "use_when": ["Topic about execution gap and follow through issues."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S26",
  "name": "Confidence Is Preparedness",
  "use_when": ["Topic about preparation as the real source of confidence."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S27",
  "name": "Why Your Team Does Not Change",
  "use_when": ["Topic about failed change efforts or repeated patterns in teams."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
```

```
},
{
  "id": "S28",
  "name": "Power Of One Good Question",
  "use_when": ["Topic about questioning, discovery, and deeper thinking."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S29",
  "name": "Your Buyer Never Forgets This One Thing",
  "use_when": ["Topic about a single behavior or moment that sticks in buyer memory."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S30",
  "name": "What Happens When You Ignore Friction",
  "use_when": ["Topic about ignoring friction, obstacles, or small blockers."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},

{
  "id": "S31",
  "name": "Your Belief Is Costing You",
  "use_when": ["Topic about harmful beliefs or mental models."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
```

```
{
  "id": "S32",
  "name": "Lesson From A Lost Deal",
  "use_when": ["Topic about losing something important and what it taught."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S33",
  "name": "Real Reason You Feel Burned Out",
  "use_when": ["Topic about burnout, overload, or emotional drain."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S34",
  "name": "Why Speed Without Clarity Fails",
  "use_when": ["Topic about rushing without clear thinking or direction."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S35",
  "name": "Your Team Is Not Lazy They Are Lost",
  "use_when": ["Topic about misreading teams and mislabeling them as lazy."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S36",
```

```
  "name": "Truth Hidden Behind A Simple Question",
  "use_when": ["Topic about deeper meaning inside a simple question or phrase."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S37",
  "name": "When You Think You Are Listening But You Are Not",
  "use_when": ["Topic about poor listening or fake listening."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S38",
  "name": "Worst Time To Solve A Problem",
  "use_when": ["Topic about timing, reactive behavior, and late action."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S39",
  "name": "Buyer Does Not Care About Your Product",
  "use_when": ["Topic about buyer focus on their own risk and goals, not features."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S40",
  "name": "Confidence Is Built Before The Call",
```

```
    "use_when": ["Topic about prep and practice before performance moments such as calls or demos."],
    "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
  },

  {
    "id": "S41",
    "name": "Question You Avoid Is The One You Need",
    "use_when": ["Topic about avoided questions, reflection, and truth facing."],
    "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
  },
  {
    "id": "S42",
    "name": "Stop Fixing Symptoms",
    "use_when": ["Topic about fixing root causes instead of surface problems."],
    "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
  },
  {
    "id": "S43",
    "name": "Team Reacts To Your Energy Not Your Words",
    "use_when": ["Topic about energy, presence, and emotional tone in leadership."],
    "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
  },
  {
    "id": "S44",
    "name": "What Happens When You Stop Chasing Validation",
```

```
  "use_when": ["Topic about validation seeking, self worth, and internal focus."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S45",
  "name": "Gap Between Busy And Useful",
  "use_when": ["Topic about activity vs impact and fake busyness."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S46",
  "name": "What Makes A Rep Truly Dangerous",
  "use_when": ["Topic about high performing, high leverage performers or reps."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S47",
  "name": "Why Founders Lose Deals They Should Win",
  "use_when": ["Topic about founder led sales and their unique mistakes."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
},
{
  "id": "S48",
  "name": "One Pattern You Must Break Early",
  "use_when": ["Topic about early career habits and patterns to break soon."],
  "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
```

```
    },
    {
      "id": "S49",
      "name": "Buyers Trust Behavior Not Language",
      "use_when": ["Topic about alignment between words and behavior, trust signals."],
      "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
    },
    {
      "id": "S50",
      "name": "What You Allow Becomes Your Culture",
      "use_when": ["Topic about standards, tolerance, and culture formation."],
      "sections": ["hook", "context", "insight", "story", "consequence", "shift", "close"]
    }
  ]
 },
 "hook_bank": {
  "description": "Hooks grouped by doctrine themes. LLM can pick from these or generate new ones with the same style.",
  "gutpunch": [
    "The real problem is never the first one you name.",
    "You think you are clear. You are not.",
    "The mistake feels small. The cost is huge.",
    "You do not lose deals. You lose moments.",
    "Buyers hear your fear before your pitch."
  ],
  "revsync": [
```

"If the buyer is confused, the deal is already gone.",

"You cannot guide a buyer if you cannot guide their risk.",

"Your process is not broken. Your assumptions are.",

"Every stalled deal started with one unclear question."

],

"competitive": [

"Your competitor is not your enemy. Your framing is.",

"You think you are competing on features. You are competing on fear.",

"A buyer chooses the vendor they understand fastest."

],

"discovery": [

"The best discovery question is the one you avoid.",

"You do not need more questions. You need better timing.",

"If the buyer is not thinking deeper, you are not selling."

],

"negotiation": [

"You never negotiate price. You negotiate confidence.",

"A cheap deal feels expensive when trust is low."

],

"leadership": [

"Teams do not copy your words. They copy your nerves.",

"Clarity reduces half your problems. Ownership reduces the rest.",

"Your silence is louder than your direction."

],

"enablement": [

"Training fails when the rep understands the lesson but not the cost.",

```
    "You cannot scale skill without shared language.",
    "Reps do not fear new tools. They fear hidden expectations."
  ],
  "culture": [
    "What you tolerate today becomes your culture tomorrow.",
    "Busy looks good. Useful wins.",
    "Consistency beats talent."
  ],
  "personal_growth": [
    "Your future changes when your questions change.",
    "You do not rise to your goals. You fall to your habits."
  ]
 },
 "selection_logic": {
  "description": "Rules for choosing post type and skeleton. Use inputs if present. Stay agnostic if missing.",
  "post_type_selection": {
    "use_inputs": [
      "If tone is sharp, prefer PT_JOLT.",
      "If user mentions lesson learned or story, prefer PT_NARRATIVE.",
      "If user wants to explain or break down something, prefer PT_INSIGHT.",
      "If user clearly challenges a popular belief, prefer PT_CONTRARIAN.",
      "If user wants to teach how to do something, prefer PT_TEACHING."
    ],
    "fallback": "If still unclear, use PT_INSIGHT."
  },
```

```
  "skeleton_selection": {
    "use_persona_if_present": [
      "If persona is SDR or AE, prefer skeletons: S12, S16, S17, S18, S29, S39, S40, S46.",
      "If persona is CRO or VP Sales, prefer skeletons: S21, S22, S26, S27, S35, S45, S50.",
      "If persona is Founder, prefer skeletons: S05, S07, S25, S33, S36, S47.",
      "If persona is Product, prefer skeletons: S12, S24, S28, S30.",
      "If persona is Marketing, prefer skeletons: S07, S08, S28, S44."
    ],
    "use_industry_if_present": [
      "If industry is SaaS, prefer skeletons: S01, S04, S17, S19, S29, S39.",
      "If industry is Travel, prefer skeletons: S10, S12, S20, S30.",
      "If industry is Retail or Commerce, prefer skeletons: S02, S12, S28, S29, S30.",
      "If industry is Fintech, prefer skeletons: S22, S31, S33, S42.",
      "If industry is Manufacturing, prefer skeletons: S24, S30, S38, S42."
    ],
    "use_tone_if_present": [
      "If tone is sharp, prefer skeletons: S01, S02, S03, S07, S13.",
      "If tone is reflective, prefer skeletons: S04, S05, S10, S14, S33.",
      "If tone is neutral, any skeleton based on topic type is acceptable."
    ],
    "fallback": "If persona, industry and tone are not provided or do not map clearly, pick from universal skeletons: S04, S05, S07, S10, S31, S41, S45."
  }
 },
 "adaptation_logic": {
  "description": "How to adapt sections when persona, industry, geography, language, or challenge are present.",
```

"persona": {

"if_present": "Adjust examples, consequences, and scenes to fit that role.",

"if_missing": "Use general work situations that any professional can relate to."

},

"industry": {

"if_present": "Use realistic scenarios and tension points from that industry only. Avoid jargon unless commonly known.",

"if_missing": "Use generic situations like meetings, calls, decisions, misalignment, or lack of clarity."

},

"geography": {

"if_present": "US and EU can be more direct. India and APAC should be firm yet polite. LatAm can be slightly warmer and relational.",

"if_missing": "Use globally neutral English without local slang or region specific references."

},

"language": {

"if_en": "Use simple, conversational global English.",

"if_en_hinglish": "Use short lines, sprinkle natural Indian English phrasing, keep grammar forgiving but clear.",

"if_es": "Keep tone warm and smooth. Use short, clear Spanish sentences.",

"if_missing": "Default to simple English as in if_en."

},

"challenge": {

"if_present": "Shape the context, story, consequences, and shift to address that specific challenge.",

"if_missing": "Keep the problem and solution generic enough to apply to many situations."

}

```
  },
  "generation_steps": {
    "description": "Ordered steps the LLM should follow.",
    "steps": [
      "Step 1: Read all provided inputs without changing them.",
      "Step 2: Decide post type using topic, tone, persona if available, else default to PT_INSIGHT.",
      "Step 3: Choose one skeleton using persona, industry, tone, or fallback rules.",
      "Step 4: Choose a hook from hook_bank that fits the topic and doctrine, or generate a new hook in the same style.",
      "Step 5: Fill sections in this order: hook, context, insight, story, consequence, shift, close.",
      "Step 6: Apply adaptation_logic for persona, industry, geography, language, and challenge only if those inputs exist.",
      "Step 7: If an input is missing, keep that dimension agnostic and universal.",
      "Step 8: Keep sentences short and conversational. Avoid jargon, buzzwords, and clichés.",
      "Step 9: Use line breaks to make the post easy to scan. One idea per line.",
      "Step 10: Return only the final post text, not the internal decision data, unless explicitly asked for it."
    ]
  },
  "output_format": {
    "description": "Target output structure.",
    "structure": [
      "Hook on its own line.",
      "Blank line.",
      "Context as 2 to 4 short lines.",
      "Blank line.",
      "Insight as 2 to 4 short lines.",
```

```
    "Blank line.",
    "Story as 3 to 8 short lines.",
    "Blank line.",
    "Consequence as 2 to 4 short lines.",
    "Blank line.",
    "Shift as 1 to 3 short lines.",
    "Blank line.",
    "Close as 1 or 2 short lines."
  ],
  "rules": [
    "No emojis unless explicitly requested.",
    "No hashtags unless explicitly requested.",
    "No call to action for likes or follows.",
    "Reflective question or line as the close.",
    "No m dash character. Use simple punctuation."
  ]
 },
 "examples": [
  {
    "name": "Example 1 - SDR Jolt Post (SaaS, PT_JOLT, S17)",
    "inputs": {
      "topic": "SDRs losing meetings in the first 30 seconds of the call",
      "core_insight": "Most SDRs lose the call before the buyer even understands why they are on the phone.",
      "persona": "SDR",
      "industry": "SaaS",
```

```
    "geography": "India",
    "tone": "sharp",
    "language": "en",
    "challenge": "cold call openers and nerves"
  },
  "selected": {
    "post_type": "jolt",
    "template_id": "T_JOLT",
    "skeleton_id": "S17",
    "hook_used": "You do not lose deals. You lose moments."
  },
  "post": "You do not lose deals. You lose moments.\n\nThink about the last cold call you made.\nYou said your name.\nYou said your company.\nYou said your title.\nAnd by the time you reached the point, the buyer had already checked out.\n\nThe real problem is not your script.\nThe real problem is the first 15 seconds where the buyer decides if you are worth their attention.\nMost SDRs use that time to talk about themselves.\nBuyers use that same time to decide if they want to stay or escape.\n\nI watched a rep do this three calls in a row.\nEvery buyer sounded polite.\nEvery buyer also rushed to end the call.\nThe rep blamed the market.\nThe truth was simpler.\nThe opener never gave the buyer a reason to think.\n\nThe cost is not just one missed meeting.\nIt is a week of dials that never turn into real conversations.\nIt is a calendar full of activity and empty of impact.\nOver time, it turns into self doubt, then low confidence, then quiet quitting on the phone.\n\nStart here instead.\nUse the first line to name a problem your buyer already feels.\nMake them pause.\nMake them think.\nThen earn the right to explain who you are.\n\nNext time you pick up the phone, ask yourself one question.\nIn my first line, am I talking about me, or am I talking about a problem they cannot ignore."
},
{
  "name": "Example 2 - CRO Insight Post (PT_INSIGHT, S22)",
  "inputs": {
```

```
  "topic": "Over reliance on vanity sales metrics",
  "core_insight": "The safest looking numbers often hide the deepest risk.",
  "persona": "CRO",
  "industry": "SaaS",
  "geography": "US",
  "tone": "insight",
  "language": "en"
 },
 "selected": {
  "post_type": "insight",
  "template_id": "T_INSIGHT",
  "skeleton_id": "S22",
  "hook_used": "Metrics everyone trusts are often the ones that hurt you most."
 },
 "post": "Metrics everyone trusts are often the ones that hurt you most.\n\nOn paper, your funnel looks fine.\nTop of funnel is healthy.\nOpportunities look stable.\nWin rate has not collapsed.\nNothing screams danger.\n\nHere is the trap.\nThe more comfortable the dashboard looks, the less likely you are to ask hard questions.\nYou look at conversion.\nYou look at pipeline coverage.\nYou do not look at how many deals die for the same three reasons every quarter.\n\nI sat with a leadership team that was proud of their numbers.\nThen we pulled one simple view.\nDeals that reached late stage and still slipped.\nThe pattern was obvious.\nNo clear decision process.\nNo risk map.\nNo real champion.\nTheir best looking metrics hid their worst execution habits.\n\nThe cost is not this quarter alone.\nIt is the story your team tells themselves.\nAs long as the dashboard looks green, they believe the system works.\nThey do not feel the need to fix discovery, tighten qualification, or coach deeper on deals.\nSo the leak continues, just below the line where you usually look.\n\nThe shift is simple.\nTreat metrics as starting points, not verdicts.\nAny number that looks too safe deserves more questions, not fewer.\nLook for repeat patterns in losses, not just volume in wins.\n\nNext time your report looks clean, ask yourself.\nWhat pain is this view hiding from me right now."
 },
```

```
  {
    "name": "Example 3 - Agnostic Narrative Post (Minimal Inputs, PT_NARRATIVE, S04)",
    "inputs": {
      "topic": "People thinking they are clear when they are not"
    },
    "selected": {
      "post_type": "insight",
      "template_id": "T_INSIGHT",
      "skeleton_id": "S04",
      "hook_used": "You think you are clear. You are not."
    },
    "post": "You think you are clear. You are not.\n\nYou give instructions and feel you have explained everything.\nThe other person nods.\nYou both leave the conversation feeling like things are aligned.\nThen a week later, the outcome looks nothing like what you had in mind.\n\nThis does not happen because people are careless.\nIt happens because clarity in your head does not mean clarity in theirs.\nWe mistake our own certainty for shared understanding.\nWe speak in short labels and shortcuts and assume others see the same picture.\n\nI once watched two people work hard on the same task.\nBoth were sincere.\nBoth were skilled.\nThey just carried two different mental images of the goal.\nNo amount of effort could fix what a simple clarifying question could have fixed at the start.\n\nThe cost is quiet.\nTime gets wasted.\nTrust slowly erodes.\nPeople start to double guess themselves or stop taking initiative.\nSoon, everyone plays safe instead of playing to win.\n\nThe shift is gentle.\nInstead of asking, \"Is this clear?\" ask, \"Can you walk me through how you are picturing this?\".\nWhen the other person explains it back in their own words, you finally see if your pictures match.\n\nBefore your next important conversation, remember.\nClarity is not how you feel when you speak.\nClarity is what the other person can do after you are done speaking."
  }
 ]
}
```

1) Developer Implementation Note

How a dev would plug this JSON spec into an LLM flow

Think of three layers:

1. **Spec Loader**

2. **Planner** (selects type + skeleton + template)

3. **Generator** (writes the post)

Step 1: Load the spec

- Store the big JSON spec as a **static config** (file, DB, or code constant).

- On app start, load it into memory: spec = load_json("prakash_linkedin_spec.json").

Step 2: Accept user inputs

Your API or app will receive something like:

```
{
  "topic": "coaching reps without killing their confidence",
  "persona": "Sales Manager",
  "industry": "SaaS",
  "tone": "reflective",
  "language": "en"
}
```

Pass this as user_inputs.

Step 3: Planner step (can be a light function or a first LLM call)

Planner responsibilities:

1. **Choose post_type** from spec.post_types

2. **Choose skeleton_id** from spec.skeletons.items using selection_logic

3. **Choose template_id** from spec.templates.items based on post_type

4. **Choose hook** from hook_bank or let LLM generate a new one

You can do this in code or inside the LLM.

Example planner output:

```
{
  "post_type": "insight",
  "skeleton_id": "S27",
  "template_id": "T_INSIGHT",
  "hook": "Teams do not copy your words. They copy your nerves."
}
```

Step 4: Generator step (LLM call)

Give the LLM:

- The **relevant slices** of the spec (not entire spec if token size is a concern)

- The user_inputs

- The planner output (chosen type, skeleton, template, hook)

- The **runtime prompt** (section 2 below)

LLM returns the post text only.

Step 5: Optionally log

- Watch which skeletons and hooks get used most

- Add your own overrides or weights later

That is it.
No extra orchestration needed beyond: **load spec → plan → generate → return text**.

Runtime Prompt (for the LLM at generation time)

You will pass this as the **system message** (or main instruction), along with:

- spec_slice (post_types, templates, skeletons, hook_bank, selection_logic, adaptation_logic, output_format)

- user_inputs

- Planner choices if done outside the LLM

You can adapt names as you like.

You generate LinkedIn posts using a fixed spec.

You receive:

- A config object called SPEC that defines post types, templates, skeletons, hooks, selection logic, adaptation logic, and output format.

- A JSON object called USER_INPUTS with optional fields:

  - topic (string, required)

  - core_insight (string, optional)

  - persona (string, optional)

  - industry (string, optional)

  - geography (string, optional)

  - tone (string, optional)

  - language (string, optional, en / en_hinglish / es)

  - challenge (string, optional)

Rules for inputs:

- Use every input that is present.

- Do not ask for missing inputs unless the user explicitly requested input collection.

- For missing inputs, remain agnostic and use neutral, universal examples.

Post type selection:

- Choose a post type from SPEC.post_types using:

  - tone if present

  - persona if present

  - topic and core_insight

- If still unclear, default to the INSIGHT type.

Skeleton selection:

- Choose one skeleton from SPEC.skeletons.items using SPEC.selection_logic.skeleton_selection.

- Use persona, industry, and tone only if provided.

- If nothing matches clearly, use one of the universal skeletons from the fallback list.

Template selection:

- Use the template whose post_type matches the chosen post type.

Hook selection:

- Use SPEC.hook_bank to pick a hook that fits the topic and doctrine, or create a new hook in the same style.

- Hook must be 1 line, sharp, and easy to understand.

Adaptation:

- If persona is present, adapt context, story, and consequence to that role.

- If industry is present, anchor scenes and tensions to that industry.

- If geography is present, adapt tone (US/EU more direct, India/APAC firm but polite, LatAm warmer).

- If language is en_hinglish, use natural Indian English phrases and short lines.

- If language is es, output in clear Spanish.

- If language is missing, use simple global English.

- If challenge is present, let it shape the problem, story, and consequence.

- If any of these fields are missing, keep that dimension universal and neutral.

Structure:

Always write the post in this order:

1) Hook

2) Context

3) Insight

4) Story

5) Consequence

6) Shift

7) Close

Output formatting:

- Hook on its own line.

- Use blank lines between sections.

- Use short lines and one idea per line.

- No emojis unless user asks.

- No hashtags unless user asks.

- No calls to action for likes or follows.

- Close with a reflective line or question.

- Do not use the long dash character. Use simple punctuation only.

Voice and style:

- Sound like a human speaking to one reader.

- Use simple, direct language.

- Avoid clichés, buzzwords, and jargon.

- Be specific, not vague.

- Focus on clarity and consequence.

Return:

- Return only the final post text.

- Do not return internal reasoning or config.

QA Checklist To Score Generated Posts

You can use this as a manual review list or turn it into an LLM-based scorer.

Score each on 1–5, then decide a pass threshold (for example, 4+ on all critical items).

A. Structural Checks

1. **Correct sections present**
    - Hook, Context, Insight, Story, Consequence, Shift, Close are all clearly present.
    - Line breaks roughly follow the spec.
2. **Hook quality**
    - One line only
    - Clear, sharp, no fluff
    - Tied to the topic
3. **Context clarity**
    - Scene is understandable in 2–4 lines

- No confusing setup

4. **Insight depth**

    - Explains a pattern or truth, not a generic statement

    - Reader can tell what changed in thinking

5. **Story usefulness**

    - Short and concrete

    - Easy to picture

    - Supports the insight, not random

6. **Consequence power**

    - Real cost is named (emotional and/or business)

    - Feels believable and specific

7. **Shift clarity**

    - One clear shift, not 5 tips
    - Reader could act on it the same day

8. **Close effectiveness**

    - Ends with a question or reflection
    - No “like / comment / follow” CTA

B. Input and Adaptation Checks

9. **Persona alignment** (if persona was given)

    - Examples feel natural for that role
    - Language and tension match their reality

10. **Industry alignment** (if industry was given)

- Examples and scenes match that industry

- No obvious mismatch (eg airlines terms in a bank post)

11. **Geography adaptation** (if geography given)

    - Directness, politeness or warmth feels right for that region

12. **Language correctness**

    - If English: simple, global, clear

    - If Hinglish: natural Indian English, spoken style

    - If Spanish: correct grammar, simple phrasing

13. **Challenge focus** (if challenge given)

    - The main problem in the post matches the challenge

    - The shift actually addresses that challenge

14. **Agnostic behavior** (if some inputs missing)

    - No random assumptions on persona or industry
    - Examples feel general, not misplaced

C. Style and Voice Checks

15. **No jargon / clichés**

    - Avoided generic phrases like “in today’s world” or “game changer”
    - No heavy corporate speak

16. **Sentence simplicity**

    - Short, clear sentences
    - Easy to read in one pass

17. **Tone consistency**

- Matches requested tone (sharp, reflective, teaching, contrarian, neutral)

- No sudden shifts in mood

18. **Consequence anchoring**

- Post does not stay at theory level

- Shows what happens if nothing changes

19. **Emotional honesty**

- Feels sincere, not fake motivational content

20. **No forbidden formatting**

- No long dash character

- No emojis or hashtags unless explicitly requested

1. postgen.py – minimal CLI wrapper

```
#!/usr/bin/env python3
import argparse
import json
import os
from openai import OpenAI

def load_spec(path: str = "prakash_linkedin_spec.json") -> dict:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def build_user_inputs(args: argparse.Namespace) -> dict:
    data = {"topic": args.topic}
    optional_fields = [
        "core_insight",
        "persona",
        "industry",
        "geography",
        "tone",
        "language",
        "challenge",
    ]
    for field in optional_fields:
        value = getattr(args, field, None)
        if value:
            data[field] = value
```

return data

RUNTIME_PROMPT = """

You generate LinkedIn posts using a fixed spec.

You will be given a JSON object that contains:

- SPEC: the configuration for post types, templates, skeletons, hooks, selection and adaptation logic, and output format.

- USER_INPUTS: optional fields with information about topic, core_insight, persona, industry, geography, tone, language, and challenge.

Rules for inputs:

- Use all fields that exist in USER_INPUTS.

- Do not ask for any missing inputs.

- For missing fields, keep that dimension neutral and universal.

Post type selection:

- Choose a post type from SPEC.post_types using tone, persona, topic, and core_insight when provided.

- If no clear choice, default to the INSIGHT type.

Skeleton selection:

- Choose one skeleton from SPEC.skeletons.items using SPEC.selection_logic.skeleton_selection.

- Use persona, industry, and tone only if provided.

- If nothing matches clearly, use one universal skeleton from the fallback list.

Template selection:

- Use the template whose post_type matches the chosen post type from SPEC.templates.

Hook selection:

- Use SPEC.hook_bank to pick a hook that fits the topic and doctrine, or create a new hook in the same style.

- Hook must be a single short line that is clear and sharp.

Adaptation:

- If persona exists, adapt context, story, and consequence to that role.

- If industry exists, adapt scenes and tensions to that industry.

- If geography exists, adapt tone for that region.

- If language is en_hinglish, use natural Indian English phrasing.

- If language is es, output in simple Spanish.

- If language is not provided, use simple global English.

- If challenge exists, let it shape the problem, story, and consequence.

- If any of these fields are missing, keep that aspect neutral and universal.

Structure of the post:

- Sections in this order: Hook, Context, Insight, Story, Consequence, Shift, Close.

Formatting:

- Hook on its own line.

- Use blank lines between sections.

- Use short lines with one idea per line.

- No emojis unless the user explicitly asks.

- No hashtags unless the user explicitly asks.

- No calls to action for likes or follows.

- Close with a reflective line or question.

- Do not use the long dash character. Use normal punctuation instead.

Voice:

- Sound like a human talking to one reader.

- Use clear, simple language.

- Avoid clichés, buzzwords, and jargon.

- Focus on clarity and consequence.

Return only the final post text. Do not return internal reasoning or JSON.

""".strip()

```
def main():
    parser = argparse.ArgumentParser(
        description="Generate a Prakash-style LinkedIn post from a topic using the spec."
    )

    parser.add_argument("topic", help="Short description of what the post should be about.")

    parser.add_argument("--core-insight", dest="core_insight", help="Main idea or lesson.", default=None)
    parser.add_argument("--persona", help="Target reader role, for example SDR, AE, CRO, Founder.", default=None)
    parser.add_argument("--industry", help="Industry, for example SaaS, Travel, Retail, Fintech.", default=None)
```

```
    parser.add_argument("--geography", help="Geo code, for example US, EU, India, APAC, LatAm.", default=None)
    parser.add_argument("--tone", help="Tone such as sharp, reflective, teaching, contrarian, neutral.", default=None)
    parser.add_argument("--language", help="en, en_hinglish, or es.", default=None)
    parser.add_argument("--challenge", help="Specific problem to target, for example demo nerves.", default=None)
    parser.add_argument("--spec-path", help="Path to the JSON spec file.", default="prakash_linkedin_spec.json")

    args = parser.parse_args()

    spec = load_spec(args.spec_path)
    user_inputs = build_user_inputs(args)

    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise RuntimeError("Please set OPENAI_API_KEY in your environment.")

    client = OpenAI(api_key=api_key)

    payload = {
        "SPEC": spec,
        "USER_INPUTS": user_inputs,
    }

    messages = [
```

```
        {"role": "system", "content": RUNTIME_PROMPT},
        {"role": "user", "content": json.dumps(payload)},
    ]

    completion = client.chat.completions.create(
        model="gpt-5.1-thinking",  # replace with your deployed model name
        messages=messages,
        temperature=0.6,
    )

    post = completion.choices[0].message.content.strip()
    print(post)

if __name__ == "__main__":
    main()
```

---

2. Make it feel like a real CLI

From the folder where this lives:

chmod +x postgen.py

Then either:

- Call it directly:

./postgen.py "Reps think they are coachable, but they avoid real feedback"

Or create a shell alias in your .bashrc or .zshrc:

alias postgen="python /full/path/to/postgen.py"

Then you can do:

postgen "Why most enablement fails after the first 30 days"

postgen "Founders losing deals they should win" --persona "Founder" --industry "SaaS" --tone "sharp"

postgen "Discovery calls that feel like a paid session" --persona "AE" --industry "SaaS" --challenge "shallow discovery"

Each call:

- Loads your spec
- Uses your selection logic
- Adapts for any inputs you pass
- Spits out a ready to post LinkedIn piece in your structure.

debug version.

Below is postgen_debug.py which:

- Uses the **same JSON spec**

- Uses almost the **same runtime logic**

- Asks the model to return a **JSON object**:

```
{
  "post": "final post text here",
  "meta": {
    "post_type": "...",
    "skeleton_id": "...",
    "template_id": "...",
    "hook": "..."
  }
}
```

The script:

- Parses this JSON

- Prints the **post**

- Then prints a small **meta** section so you can see what the LLM chose

---

postgen_debug.py

```
#!/usr/bin/env python3
import argparse
import json
import os
from openai import OpenAI


def load_spec(path: str = "prakash_linkedin_spec.json") -> dict:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)


def build_user_inputs(args: argparse.Namespace) -> dict:
    data = {"topic": args.topic}
    optional_fields = [
        "core_insight",
        "persona",
        "industry",
        "geography",
        "tone",
        "language",
        "challenge",
    ]
    for field in optional_fields:
```

```
        value = getattr(args, field, None)
        if value:
            data[field] = value
    return data

RUNTIME_PROMPT_DEBUG = """
```

You generate LinkedIn posts using a fixed spec and you must return a JSON object.

You will be given a JSON object that contains:

- SPEC: the configuration for post types, templates, skeletons, hooks, selection and adaptation logic, and output format.

- USER_INPUTS: optional fields with information about topic, core_insight, persona, industry, geography, tone, language, and challenge.

Rules for inputs:

- Use all fields that exist in USER_INPUTS.

- Do not ask for any missing inputs.

- For missing fields, keep that dimension neutral and universal.

Post type selection:

- Choose a post type from SPEC.post_types using tone, persona, topic, and core_insight when provided.

- If no clear choice, default to the INSIGHT type.

Skeleton selection:

- Choose one skeleton from SPEC.skeletons.items using SPEC.selection_logic.skeleton_selection.

- Use persona, industry, and tone only if provided.

- If nothing matches clearly, use one universal skeleton from the fallback list.

Template selection:

- Use the template whose post_type matches the chosen post type from SPEC.templates.

Hook selection:

- Use SPEC.hook_bank to pick a hook that fits the topic and doctrine, or create a new hook in the same style.

- Hook must be a single short line that is clear and sharp.

Adaptation:

- If persona exists, adapt context, story, and consequence to that role.

- If industry exists, adapt scenes and tensions to that industry.

- If geography exists, adapt tone for that region.

- If language is en_hinglish, use natural Indian English phrasing.

- If language is es, output in simple Spanish.

- If language is not provided, use simple global English.

- If challenge exists, let it shape the problem, story, and consequence.

- If any of these fields are missing, keep that aspect neutral and universal.

Structure of the post:

- Sections in this order: Hook, Context, Insight, Story, Consequence, Shift, Close.

Formatting:

- Hook on its own line.

- Use blank lines between sections.

- Use short lines with one idea per line.
- No emojis unless the user explicitly asks.
- No hashtags unless the user explicitly asks.
- No calls to action for likes or follows.
- Close with a reflective line or question.
- Do not use the long dash character. Use normal punctuation instead.

Voice:
- Sound like a human talking to one reader.
- Use clear, simple language.
- Avoid clichés, buzzwords, and jargon.
- Focus on clarity and consequence.

Return format:
- Return a valid JSON string with this structure:

```
{
  "post": "<final post text only>",
  "meta": {
    "post_type": "<chosen post type id or name>",
    "skeleton_id": "<chosen skeleton id>",
    "template_id": "<chosen template id>",
    "hook": "<final hook line used>"
  }
}
```

Important:

- Do not include any explanation outside this JSON object.

- Do not wrap the JSON in code fences.

- Ensure the JSON is syntactically valid.

""".strip()

def main():

    parser = argparse.ArgumentParser(

        description="Generate a Prakash-style LinkedIn post with debug meta from a topic using the spec."

)

    parser.add_argument("topic", help="Short description of what the post should be about.")

    parser.add_argument("--core-insight", dest="core_insight", help="Main idea or lesson.", default=None)

    parser.add_argument("--persona", help="Target reader role, for example SDR, AE, CRO, Founder.", default=None)

    parser.add_argument("--industry", help="Industry, for example SaaS, Travel, Retail, Fintech.", default=None)

    parser.add_argument("--geography", help="Geo code, for example US, EU, India, APAC, LatAm.", default=None)

    parser.add_argument("--tone", help="Tone such as sharp, reflective, teaching, contrarian, neutral.", default=None)

    parser.add_argument("--language", help="en, en_hinglish, or es.", default=None)

    parser.add_argument("--challenge", help="Specific problem to target, for example demo nerves.", default=None)

```
    parser.add_argument("--spec-path", help="Path to the JSON spec file.",
default="prakash_linkedin_spec.json")

    args = parser.parse_args()

    spec = load_spec(args.spec_path)
    user_inputs = build_user_inputs(args)

    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise RuntimeError("Please set OPENAI_API_KEY in your environment.")

    client = OpenAI(api_key=api_key)

    payload = {
        "SPEC": spec,
        "USER_INPUTS": user_inputs,
    }

    messages = [
        {"role": "system", "content": RUNTIME_PROMPT_DEBUG},
        {"role": "user", "content": json.dumps(payload)},
    ]

    completion = client.chat.completions.create(
        model="gpt-5.1-thinking",  # replace with your deployed model name
```

```
        messages=messages,
        temperature=0.6,
    )

    raw = completion.choices[0].message.content.strip()

    try:
        data = json.loads(raw)
    except json.JSONDecodeError:
        print("Could not parse model output as JSON. Raw output below:\n")
        print(raw)
        return

    post = data.get("post", "").strip()
    meta = data.get("meta", {})

    print("\n===== LINKEDIN POST =====\n")
    print(post)
    print("\n===== META =====\n")
    print(f"Post type   : {meta.get('post_type')}")
    print(f"Skeleton ID : {meta.get('skeleton_id')}")
    print(f"Template ID : {meta.get('template_id')}")
    print(f"Hook        : {meta.get('hook')}")

if __name__ == "__main__":
    main()
```

How to use

Same pattern as the non debug version.

chmod +x postgen_debug.py

./postgen_debug.py "Why most enablement dies after onboarding"

```
./postgen_debug.py \
  "Founders losing deals they should win" \
  --persona "Founder" \
  --industry "SaaS" \
  --tone "sharp"
```

Output shape:

- First the full post

- Then a tiny meta block telling you:
    - Which **post type** it used
    - Which **skeleton**
    - Which **template**
    - Which **hook** line